Test Cases and Testing Results

for

FamPlan

Version 1.0 – Lab 4 Testing Deliverables

Prepared by Banana Five-O

NTU SC2006 Software Engineering

21 October 2026

Work product	Author	Username
Test cases, test code and testing results	MUHAMMAD ILHAM BIN IRWAN	MUHA0313

Version	Date	Description
1.0	21 October 2026	Black-box and white-box test cases with execution results for Lab 4 §3.2

Contents

1. Introduction 3
1.1 Purpose and scope 3
1.2 Test environment 3
2. Black-box Testing: ScoringController 3
2.1 Class under test 3
2.2 Equivalence classes 4
2.3 Boundary values 4
2.4 Test cases and results 5
2.5 Minimising the number of test cases 6
3. White-box Testing: Basis Path Testing 7
3.1 WB1: ScoringController.calculateScore() 7
3.2 WB2: dedupeFacilities() 9
3.3 Checking that the tests can fail 11
4. Test Execution Summary 11
4.1 Additional tests for NFR-MAINT-02 11
5. Traceability: Requirements to Tests 12

1. Introduction

1.1 Purpose and scope

This document records the test cases and testing results required by Lab 4 §3.2. It covers:

- Black-box testing (§3.2.1) of one important control class, ScoringController, using equivalence class partitioning and boundary value analysis.
- White-box testing (§3.2.2) of two methods with complex application logic, calculateScore() and dedupeFacilities(), using basis path testing.
- Execution of every test case and the recorded results (§3.2.4), plus additional tests and branch coverage evidence for NFR-MAINT-02.

The techniques follow Tutorial 8 (equivalence class and boundary value testing) and Tutorial 9 (control flow testing). Every requirement ID refers to FamPlan SRS v1.5.

1.2 Test environment

Item	Value
Code under test	prototype/server (branch lab-4-updates-ham)
Test framework	Vitest 5.0.2 with V8 coverage; Node.js 22.14.0; Windows 11
Database (DB-backed suites only)	PostgreSQL 16, database famplan_test
Test files	server/test/scoring.blackbox.test.ts (black-box) server/test/whitebox.test.ts (white-box) server/test/sources.test.ts, server/test/service.test.ts (NFR-MAINT-02 coverage)
Run all tests	cd prototype/server && npx vitest run
Coverage report	cd prototype/server && npx vitest run --coverage
Execution date	21 October 2026

All test cases are automated. Each test is named with the test case ID used in this document, so every row in the result tables maps to one test in the code. Expected outputs were calculated by hand from the scoring rules in SRS §6 and SCORING_CONFIG (famplan-score-v1.0) before the tests were run.

2. Black-box Testing: ScoringController

2.1 Class under test

ScoringController (server/src/scoring/score.ts, SRS Table 9.5.2) calculates the Family Suitability Score. It was chosen because it holds the core application logic the client relies on (FR-DEC-01 to FR-DEC-03), and the SRS states its rules as numeric ranges. That makes it well suited to equivalence class and boundary value testing.

Its public operation is calculateScore(evidence, weights, …). It turns each criterion's raw value into a normalised value from 0 to 1 using one of three rule types, weights the criteria, and returns a total from 0 to 100. A total ending in exactly .5 rounds upward.

Rule type	Criteria	Normalised value
nearest (distance)	groceries, healthcare (150 m – 1200 m) public transport (250 m – 1500 m)	1 at or below best; 0 at or beyond worst; linear between
count	childcare (target 30), schools (6), green spaces (8), accessibility (12)	count ÷ target, capped at 1
minutes	commute (20 min – 75 min)	1 at or below best; 0 at or beyond worst; linear between

Testing approach. One representative criterion is tested for each rule type: groceries for distance, childcare for count, and commute for minutes. Within a test only the criterion under test carries weight, so the total equals its normalised value × 100. That isolates one input variable at a time (the single-fault assumption).

Invalid inputs. While designing the invalid classes, we found that before Lab 4 calculateScore() did not reject invalid weights or measurements itself. It relied on the API schema. A guard, assertValidScoringInput(), was therefore added. It collects every invalid field and throws one ScoringInputError that names each criterion, so the control class enforces its own input domain.

2.2 Equivalence classes

Input variable	Class	Condition	Valid?	Expected behaviour
Distance d (groceries)	EC-D1	d < 0 or not a number	Invalid	ScoringInputError
	EC-D2	0 ≤ d ≤ 150	Valid	normalised = 1
	EC-D3	150 < d < 1200	Valid	normalised = (1200 − d) ÷ 1050
	EC-D4	d ≥ 1200	Valid	normalised = 0
Count c (childcare)	EC-C1	c < 0	Invalid	ScoringInputError
	EC-C2	c not a whole number	Invalid	ScoringInputError
	EC-C3	0 ≤ c < 30	Valid	normalised = c ÷ 30
	EC-C4	c ≥ 30	Valid	normalised = 1
Minutes m (commute)	EC-M1	m < 0	Invalid	ScoringInputError
	EC-M2	0 ≤ m ≤ 20	Valid	normalised = 1
	EC-M3	20 < m < 75	Valid	normalised = (75 − m) ÷ 55
	EC-M4	m ≥ 75	Valid	normalised = 0
Weight w (FR-PREF-05)	EC-W1	w < 0	Invalid	ScoringInputError
	EC-W2	w ∈ {0, 1, 2, 3, 4, 5}	Valid	used as the criterion weight
	EC-W3	w > 5	Invalid	ScoringInputError
	EC-W4	w not a whole number	Invalid	ScoringInputError
	EC-W5	weight missing for a criterion	Invalid	ScoringInputError
Data availability (FR-DEC-03)	EC-A1	every criterion has data	Valid	standard explanation
	EC-A2	a weighted criterion has no data	Valid	excluded; weight redistributed
	EC-A3	no criterion has data	Valid	total 0 with explanation
	EC-A4	data present but every weight 0	Valid	equal weighting fallback
Fraction f of unrounded total (FR-DEC-02)	EC-R1	f < 0.5	Valid	round down
	EC-R2	f = 0.5	Valid	round up
	EC-R3	f > 0.5	Valid	round up

2.3 Boundary values

Each boundary is tested just below, on, and just above it (robust boundary value analysis). Integer steps are used for distance (1 m), count (1) and minutes (1 min).

Variable	Boundary	Values tested	Test cases
Distance d	lower bound 0	−1, 0	BB-D1, BB-D2
	best = 150 m	149, 150, 151	BB-D3 – BB-D5
	worst = 1200 m	1199, 1200, 1201	BB-D7 – BB-D9
Count c	lower bound 0	−1, 0, 1	BB-C1 – BB-C3
	target = 30	29, 30, 31	BB-C5 – BB-C7
Minutes m	lower bound 0	−1	BB-M1
	best = 20 min	19, 20, 21	BB-M2 – BB-M4
	worst = 75 min	74, 75, 76	BB-M6 – BB-M8
Weight w	minimum 0	−1, 0, 1	BB-W1 – BB-W3
	maximum 5	4, 5, 6	BB-W4 – BB-W6
Rounding	.5	8.33, 12.5, 16.67	BB-R1 – BB-R3

2.4 Test cases and results

Inputs not listed are held at their nominal value (every criterion normalises to 0.5). In the D, C, M and R groups only the criterion under test is weighted (weight 5). In the W group, childcare = 30 (normalised 1) and schools = 0 (normalised 0) with schools weight 5, so the total is w ÷ (w + 5) × 100. Values in brackets are the unrounded totals.

ID	Test input	Class / boundary	Expected output	Actual output	Result
BB-D1	groceries = −1 m	EC-D1; BV below 0	ScoringInputError	ScoringInputError thrown	Pass
BB-D2	groceries = 0 m	EC-D2; BV 0	total 100	total 100	Pass
BB-D3	groceries = 149 m	EC-D2; BV 150−1	total 100	total 100	Pass
BB-D4	groceries = 150 m	EC-D2; BV 150	total 100	total 100	Pass
BB-D5	groceries = 151 m	EC-D3; BV 150+1	total 100; normalised 0.999048	total 100; normalised 0.999048	Pass
BB-D6	groceries = 675 m	EC-D3; nominal	total 50	total 50	Pass
BB-D7	groceries = 1199 m	EC-D3; BV 1200−1	total 0; normalised 0.000952	total 0; normalised 0.000952	Pass
BB-D8	groceries = 1200 m	EC-D4; BV 1200	total 0	total 0	Pass
BB-D9	groceries = 1201 m	EC-D4; BV 1200+1	total 0	total 0	Pass
BB-D10	groceries = NaN	EC-D1 (not a number)	ScoringInputError	ScoringInputError thrown	Pass
BB-C1	childcare = −1	EC-C1; BV below 0	ScoringInputError	ScoringInputError thrown	Pass
BB-C2	childcare = 0	EC-C3; BV 0	total 0	total 0	Pass
BB-C3	childcare = 1	EC-C3; BV 0+1	total 3 (3.33)	total 3 (3.33)	Pass
BB-C4	childcare = 15	EC-C3; nominal	total 50	total 50	Pass
BB-C5	childcare = 29	EC-C3; BV 30−1	total 97 (96.67)	total 97 (96.67)	Pass
BB-C6	childcare = 30	EC-C4; BV 30	total 100	total 100	Pass
BB-C7	childcare = 31	EC-C4; BV 30+1	total 100	total 100	Pass
BB-C8	childcare = 2.5	EC-C2	ScoringInputError	ScoringInputError thrown	Pass
BB-M1	commute = −1 min	EC-M1; BV below 0	ScoringInputError	ScoringInputError thrown	Pass
BB-M2	commute = 19 min	EC-M2; BV 20−1	total 100	total 100	Pass

ID	Test input	Class / boundary	Expected output	Actual output	Result
BB-M3	commute = 20 min	EC-M2; BV 20	total 100	total 100	Pass
BB-M4	commute = 21 min	EC-M3; BV 20+1	total 98 (98.18)	total 98 (98.18)	Pass
BB-M5	commute = 47.5 min	EC-M3; nominal	total 50	total 50	Pass
BB-M6	commute = 74 min	EC-M3; BV 75−1	total 2 (1.82)	total 2 (1.82)	Pass
BB-M7	commute = 75 min	EC-M4; BV 75	total 0	total 0	Pass
BB-M8	commute = 76 min	EC-M4; BV 75+1	total 0	total 0	Pass
BB-W1	childcare weight = −1	EC-W1; BV 0−1	ScoringInputError	ScoringInputError thrown	Pass
BB-W2	childcare weight = 0	EC-W2; BV 0	total 0	total 0	Pass
BB-W3	childcare weight = 1	EC-W2; BV 0+1	total 17 (16.67)	total 17 (16.67)	Pass
BB-W4	childcare weight = 4	EC-W2; BV 5−1	total 44 (44.44)	total 44 (44.44)	Pass
BB-W5	childcare weight = 5	EC-W2; BV 5	total 50	total 50	Pass
BB-W6	childcare weight = 6	EC-W3; BV 5+1	ScoringInputError	ScoringInputError thrown	Pass
BB-W7	childcare weight = 2.5	EC-W4	ScoringInputError	ScoringInputError thrown	Pass
BB-W8	schools weight omitted	EC-W5	ScoringInputError	ScoringInputError thrown	Pass
BB-E1	childcare weight 6, schools weight 2.5, groceries −1 m	EC-W3 + EC-W4 + EC-D1 together	One ScoringInputError naming childcare, schools and groceries	One ScoringInputError naming childcare, schools and groceries	Pass
BB-A1	all criteria at nominal (0.5), all weights 3	EC-A1	total 50; standard explanation	total 50; standard explanation	Pass
BB-A2	childcare 30, commute missing; weights childcare 5, commute 5	EC-A2	total 100; commute marked missing; “redistributed” note	total 100; commute marked missing; “redistributed” note	Pass
BB-A3	no criterion has data; childcare weight 3	EC-A3	total 0; “No criterion has sufficient data” note	total 0; “No criterion has sufficient data” note	Pass
BB-A4	childcare 30, others nominal; all weights 0	EC-A4	total 56 (56.25); “weighted equally” note	total 56 (56.25); “weighted equally” note	Pass
BB-R1	accessibility = 1 bus stop (only weighted)	EC-R1 (8.33)	total 8	total 8	Pass
BB-R2	green spaces = 1 park (only weighted)	EC-R2 (exactly 12.5)	total 13	total 13	Pass
BB-R3	accessibility = 2 bus stops (only weighted)	EC-R3 (16.67)	total 17	total 17	Pass

2.5 Minimising the number of test cases

- Single-fault assumption: each invalid class has its own test with every other input valid, so a failure points to one cause. BB-E1 is the one exception. It combines three invalid inputs on purpose, to check that every field error is reported together.
- Valid classes for different criteria are not combined. The rule types are independent, so one representative criterion per rule type covers the logic shared by the criteria of that type.
- Each nominal test (BB-D6, BB-C4, BB-M5) also serves as the equivalence class representative for its linear class, so no separate class test is needed.
- In total, 42 test cases cover 24 equivalence classes and 11 boundaries. A full combinatorial design over 8 criteria and their weights would need thousands of cases.

3. White-box Testing: Basis Path Testing

Control flow graphs follow the Tutorial 9 conventions. A compound condition such as a && b is one decision node. Library calls whose callbacks contain no decisions (filter, reduce) are treated as single process nodes. map and find are treated as loops because their callbacks contain decisions. Cyclomatic complexity is calculated as V(G) = E − N + 2 and checked against V(G) = P + 1, where P is the number of decision nodes.

3.1 WB1: ScoringController.calculateScore()

calculateScore() (server/src/scoring/score.ts, lines 68–137) applies the missing-data and all-zero-weight rules and builds the itemised breakdown (FR-DEC-01 to FR-DEC-03, NFR-TRANS-01).

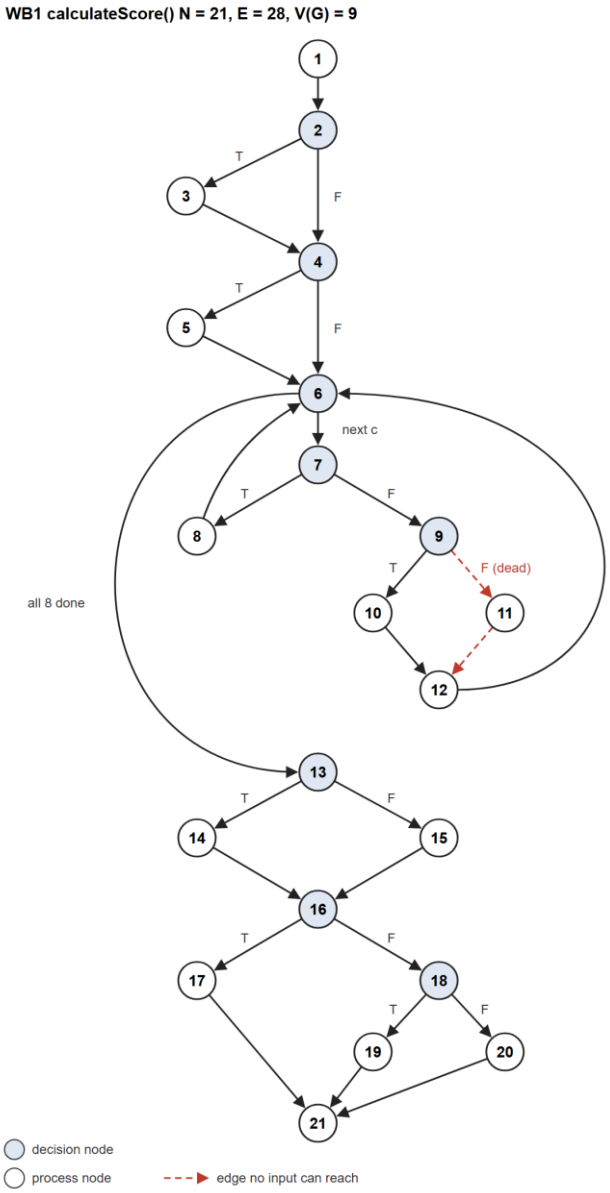


Figure 1. Control flow graph of calculateScore(). Node numbers refer to the table below.

Node	Lines	Statement / decision
1	75–79	assertValidScoringInput(); available = criteria with data; weightSum = Σ weights of available; effective = weights; notes = []
2	81	available.length > 0 && weightSum === 0 ?

Node	Lines	Statement / decision
3	82–84	equal-weight fallback: effective = 1 per available criterion; weightSum = available.length; add fallback note
4	86–88	missing, missingWeighted computed; missingWeighted.length > 0 ?
5	88	add missing-data note
6	90	CRITERIA.map: another criterion c? (always 8 iterations)
7	91–93	ev.value === null ?
8	94–105	return missing entry (effectiveWeight 0, contribution 0)
9	107–108	n = normalise(rule, value); weightSum > 0 ?
10	108	share = effective[c] ÷ weightSum
11	108	share = 0
12	109–119	return entry with contribution = share × n × 100
13	122–124	unrounded = Σ contributions; available.length === 0 ?
14	124	total = 0
15	124	total = roundHalfUp(unrounded)
16	130	available.length === 0 ?
17	131	weightTreatment = “No criterion has sufficient data …”
18	132	notes.length > 0 ?
19	133	weightTreatment = notes joined
20	134	weightTreatment = standard explanation
21	123–136	return FamilyScoreDto

Cyclomatic complexity: V(G) = E − N + 2 = 28 − 21 + 2 = 9, and V(G) = P + 1 = 8 + 1 = 9 (decision nodes 2, 4, 6, 7, 9, 13, 16, 18). The basis set therefore has 9 linearly independent paths. They were derived with the baseline method: start from a typical path, then flip one decision at a time.

Path	Node sequence	Derived by	Feasibility	Test
P1	`1-2-4-6-(7-9-10-12-6)×8-13-15-16-18-20-21`	Baseline	Feasible	WB1-P1
P2	`1-2-3-4-6-(7-9-10-12-6)×8-13-15-16-18-19-21`	Node 2 → T	Feasible	WB1-P2
P3	`1-2-4-5-6-…-(7-8-6)-…-13-15-16-18-19-21`	Node 4 → T	Feasible	WB1-P3
P4	`1-2-4-6-…-(7-8-6)-…-13-15-16-18-20-21`	Node 7 → T	Feasible	WB1-P4
P5	`1-2-4-5-6-(7-8-6)×8-13-14-16-17-21`	Node 13 → T	Feasible	WB1-P5
P6	`1-2-4-6-13-…`	Node 6 → exit at once	Infeasible	—
P7	`…-7-9-11-12-6-…`	Node 9 → F	Infeasible	—
P8	`…-13-15-16-17-21`	Node 16 → T alone	Infeasible	—
P9	`…-13-14-16-18-…`	Node 16 → F alone	Infeasible	—

(7-9-10-12-6) is one loop iteration for a criterion with data; (7-8-6) is one iteration for a criterion without data.

Why four paths are infeasible:

- P6: CRITERIA always has 8 entries, so the loop at node 6 can never exit before its first iteration.
- P7: node 9 is reached only for a criterion with data, so available.length ≥ 1. The guard ensures every weight is ≥ 0, so weightSum ≥ 0, and if weightSum is 0, node 3 sets it to available.length ≥ 1. weightSum > 0 is therefore always true, and the else branch (node 11) is dead code. The coverage report confirms it: line 108 has the only unreachable branch in calculateScore().

- P8 and P9: nodes 13 and 16 test the same condition (available.length === 0) and nothing changes in between, so the true branch of one cannot be paired with the false branch of the other.

Test cases and results:

ID	Test input	Expected output	Actual output	Result
WB1-P1	all 8 criteria at nominal (0.5); all weights 3	total 50; weightTreatment = standard explanation	total 50; weightTreatment = standard explanation	Pass
WB1-P2	all 8 criteria at nominal; all weights 0	total 50; each effectiveWeight 12.5; “weighted equally” note	total 50; each effectiveWeight 12.5; “weighted equally” note	Pass
WB1-P3	childcare 30, commute missing; weights childcare 5, commute 5	total 100; commute missing with contribution 0; “redistributed” note	total 100; commute missing with contribution 0; “redistributed” note	Pass
WB1-P4	childcare 30, schools 0, commute missing; weights childcare 5, schools 5	total 50; commute missing; standard explanation (no note)	total 50; commute missing; standard explanation (no note)	Pass
WB1-P5	no criterion has data; childcare weight 3	total 0; unroundedTotal 0; every entry missing; “No criterion …” explanation	total 0; unroundedTotal 0; every entry missing; “No criterion …” explanation	Pass

Statement and branch coverage. All 20 reachable nodes and all 26 reachable edges are covered by just three of these tests: WB1-P2, WB1-P4 and WB1-P5. Node 11 and its two edges cannot be reached by any input. The remaining basis paths add the independent-path assurance that basis path testing requires.

3.2 WB2: dedupeFacilities()

dedupeFacilities() (server/src/neighbourhoods/dedupe.ts, lines 31–48) is used by NeighbourhoodIndex to merge facility records from several data sources (NFR-DATA-04). Two records are the same facility when they share a category, their normalised names match, and they are no more than 60 m apart.

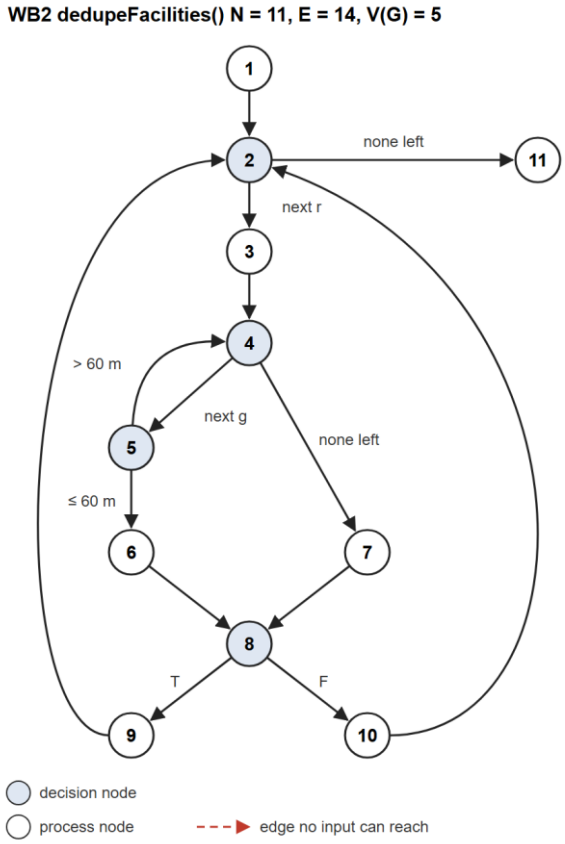


Figure 2. Control flow graph of dedupeFacilities(). Node numbers refer to the table below.

Node	Lines	Statement / decision
1	32–34	sorted = records ordered by category, sourceId, id; buckets = new Map(); groups = []
2	35	for (r of sorted): another record r?
3	36–37	key = category | normaliseName(name); candidates = buckets.get(key) ?? []
4	38	candidates.find: another candidate group g?
5	38	haversineM(g[0], r) ≤ DEDUPE_DISTANCE_M (60 m) ?
6	38	match = g
7	38	match = undefined
8	39	if (match)
9	39	match.push(r)
10	41–44	g = [r]; candidates.push(g); buckets.set(key, candidates); groups.push(g)
11	47	return groups

Cyclomatic complexity: V(G) = E − N + 2 = 14 − 11 + 2 = 5, and V(G) = P + 1 = 4 + 1 = 5 (decision nodes 2, 4, 5, 8).

Path	Node sequence	Derived by	Feasibility	Test
P1	`1-2-11`	Baseline (loop skipped)	Feasible	WB2-P1
P2	`1-2-3-4-7-8-10-2-11`	Node 2 → enter loop	Feasible	WB2-P2
P3	`1-2-3-4-7-8-10-2-3-4-5-6-8-9-2-11`	Node 4 → next g; node 5 → ≤ 60 m	Feasible	WB2-P3
P4	`1-2-3-4-7-8-10-2-3-4-5-4-7-8-10-2-11`	Node 5 → > 60 m	Feasible	WB2-P4

Path	Node sequence	Derived by	Feasibility	Test
P5	…-4-5-6-8-10-…	Node 8 → F after a match	Infeasible	—

P5 is infeasible: node 6 sets match to a non-empty group, so the decision at node 8 is always true after node 6 and always false after node 7.

Test cases and results:

ID	Test input	Expected output	Actual output	Result
WB2-P1	records = [ ]	[ ] (no groups)	[ ] (no groups)	Pass
WB2-P2	one supermarket A “Sheng Siong” at 1.3500, 103.8	[[A]]	[[A]]	Pass
WB2-P3	A “Sheng Siong Pte Ltd” at 1.3500 (src-a); B “SHENG SIONG” at 1.3502 (src-b), ≈ 22 m apart	[[A, B]] (one facility)	[[A, B]] (one facility)	Pass
WB2-P4	A and B both “Sheng Siong”, at 1.3500 and 1.3550, ≈ 556 m apart	[[A], [B]] (two facilities)	[[A], [B]] (two facilities)	Pass

Statement and branch coverage. WB2-P3 alone executes all 11 nodes (100% statement coverage). WB2-P3 and WB2-P4 together traverse all 14 edges (100% branch coverage). The coordinates were chosen far from the 60 m threshold (about 22 m and 556 m) so each test checks one path, not the threshold itself.

3.3 Checking that the tests can fail

Basis path tests written for existing code pass the first time they are run. To show they can detect faults, three deliberate faults were introduced one at a time and the white-box tests were run against each:

Fault introduced	Tests that failed
Line 81: weightSum === 0 changed to weightSum !== 0	WB1-P1, WB1-P2, WB1-P3, WB1-P4
Line 88: missingWeighted.length > 0 changed to ≥ 0	WB1-P1, WB1-P4
dedupe.ts line 38: ≤ DEDUPE_DISTANCE_M changed to >	WB2-P3, WB2-P4

Every fault was detected, and the original code was restored afterwards.

4. Test Execution Summary

The full server suite was run on 21 October 2026: 170 of 170 tests passed and none failed. That includes the 42 black-box and 9 white-box test cases in this document, the additional NFR-MAINT-02 tests in Section 4.1, and the existing API, synchronisation and SaaS tests.

4.1 Additional tests for NFR-MAINT-02

NFR-MAINT-02 requires at least 80% branch coverage for family scoring, proximity aggregation, facility deduplication and external-data transformation. The first coverage run showed that two of these were below target, so two more test files were added:

Test file	Module	Tests	What is checked	Branches before → after
sources.test.ts	synchronisation/sources.ts (external-data transformation)	13	Each of the 9 source adapters with a small sample of the provider payload: field mapping, title-casing, category classification, skipping invalid, repeated or out-of-area rows, schema errors and coverage notes. Network calls are replaced with fixed payloads.	0% → 96.8%

Test file	Module	Tests	What is checked	Branches before → after
service.test.ts	neighbourhoods/service.ts (proximity aggregation)	29	Name matching with spelling tolerance, address search (including a failed address search), the area, stage, count, distance and commute filters at their boundaries, sorting, pagination, and amenity availability states (available, no match, incomplete, unavailable).	60.5% → 93.0%

Coverage results. The 80% target is now enforced in server/vitest.config.ts (branch threshold 80, per file), so the coverage command fails if any NFR-MAINT-02 module drops below the target.

Module (NFR-MAINT-02)	Statements	Branches	≥ 80% branches
Family scoring: scoring/score.ts	100% (53/53)	95.23% (40/42)	Met
Proximity aggregation: neighbourhoods/service.ts	98.44% (127/129)	93.02% (120/129)	Met
Facility deduplication: neighbourhoods/dedupe.ts	100% (19/19)	100% (7/7)	Met
External-data transformation: synchronisation/sources.ts	99.36% (156/157)	96.79% (151/156)	Met
External-data validation: synchronisation/validate.ts	100% (45/45)	92.5% (37/40)	Met

Findings:

- Dead code at score.ts line 108: the weightSum > 0 check can never be false (path P7). It can be removed in a later refactor without changing behaviour.
- score.ts line 82: the false branch of available.includes(c) inside the equal-weight fallback is reachable (every weight 0 and some criterion missing), but it is not in the basis set, because that callback is modelled as one process node. A follow-up test is recommended.
- service.ts lines 179–180 place neighbourhoods without a sort value last. With real data this branch cannot be reached: the nearest-facility distance has no radius limit, so either every neighbourhood has a value or none does. The rule is kept as a safeguard for FR-DISC-05.
- The MOE schools adapter passes the “Nearby MRT” text through in capitals while other fields are title-cased. This is a minor display inconsistency, not a defect.
- Black-box analysis led to a design improvement: ScoringController now rejects invalid weights and measurements itself instead of relying only on the API schema.

5. Traceability: Requirements to Tests

Requirement	Design class	Implementation	Test cases
FR-PREF-05 integer weights 0–5, field errors	ScoringController	score.ts: assertValidScoringInput()	BB-W1 – BB-W8, BB-E1
FR-DEC-01 score from weights and normalisation rules	ScoringController	score.ts: calculateScore(), normalise(); config.ts	BB-D1 – BB-D10, BB-C1 – BB-C8, BB-M1 – BB-M8, WB1-P1
FR-DEC-02 itemised contributions; exact .5 rounds up	ScoringController	score.ts: roundHalfUp(), calculateScore()	BB-R1 – BB-R3, WB1-P2
FR-DEC-03 missing data disclosed, weight redistributed	ScoringController	score.ts: calculateScore()	BB-A1 – BB-A4, WB1-P3 – WB1-P5

Requirement	Design class	Implementation	Test cases
NFR-DATA-04 duplicate facilities not counted twice	NeighbourhoodIndex	dedupe.ts: dedupeFacilities()	WB2-P1 – WB2-P4
FR-DISC-01, FR-DISC-02, FR-DISC-05 search, filters and sorting	DiscoveryController	service.ts: searchNeighbourhoods(), nameMatches()	service.test.ts
FR-NBH-03, FR-NBH-06, FR-NBH-07, FR-NBH-10 nearby facilities and availability	NeighbourhoodController	service.ts: amenitySummaries(), facilitiesNear()	service.test.ts
NFR-MAINT-01 isolated source adapters	PublicDataSourceBoundary adapters	sources.ts: SOURCES[].fetch()	sources.test.ts
NFR-MAINT-02 ≥ 80% branch coverage	ScoringController, DiscoveryController, NeighbourhoodIndex, DataSyncController	scoring/, service.ts, dedupe.ts, sources.ts, validate.ts	Coverage report (Section 4.1)